

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2

За седьмой семестр

По дисциплине «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:
Студент 4 курса
Группы ИИ-26
Черноиван А.Н.
Проверил:
Хацкевич А.С.

Цель работы: практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.

Инструкция по подготовке и запуску стенда:

1. Установите Docker и Docker Compose (рекомендуемая среда для развертывания Nginx + WAF).
2. Для задания 3 вам потребуется утилита Apache Bench (ab). На Linux/WSL она устанавливается командой `sudo apt install apache2-utils`. На Windows можно использовать аналоги — утилиты `hey` или `bombardier`.
3. Перейдите в директорию стенда (где находится файл `docker-compose.yml`) и выполните команду: `docker-compose up -d`.
4. Убедитесь, что уязвимое веб-приложение доступно напрямую по порту 5000 (`http://127.0.0.1:5000`), а защищенный контур через Nginx WAF доступен по порту 80 (`http://127.0.0.1`).
5. Для выполнения всех шагов с вашего рабочего компьютера (терминала) используются консольные утилиты `curl` и `ab`.

Индивидуальные варианты

Номер варианта определяется по формуле: (Номер студента в списке журнала % 15) + 1.

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
7	ZGrab/0.x	10007	/search	5r/s	15

Задание 1 (Базовая фильтрация OWASP CRS):

1. Команду `curl` для проведения SQLi-атаки напрямую к бэкенду.

напрямую к бэкенду → ожидаем 200 OK

```
curl -i "http://127.0.0.1:5001/?id=1'%20OR%20'1'='1"
```

2. Команду `curl` для проведения SQLi-атаки через Nginx-периметр.

через WAF → ожидаем 403 Forbidden

```
curl -i "http://127.0.0.1/?id=1'%20OR%20'1'='1"
```

3. Скриншот или консольный вывод ответа Nginx, демонстрирующий блокировку атаки (403 Forbidden) базовыми правилами WAF.

```

anastasiacernoivan@MacBook-Pro-Anastasia lab2 % curl -i "http://127.0.0.1:5001/?id=1'%20OR%20'1'='1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.10.22
Date: Wed, 07 Oct 2026 06:38:28 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close

Backend OK. Endpoint reached: /%
anastasiacernoivan@M
cBook-Pro-Anastasia lab2 % curl -i "http://127.0.0.1/?id=1'%20OR%20'1'='1"
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Wed, 07 Oct 2026 06:38:50 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
anastasiacernoivan@MacBook-Pro-Anastasia lab2 % █

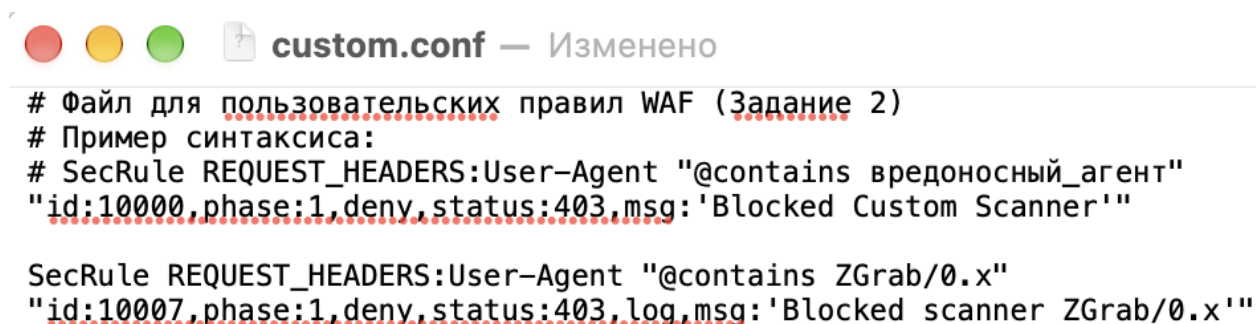
```

Задание 2 (Пользовательские правила WAF):

1. Целевой вредоносный User-Agent и ID правила из вашего варианта.

```
curl -i -H "User-Agent: ZGrab/0.x" http://127.0.0.1/
```

2. Написанный вами исходный код правила (SecRule) в файле custom.conf для блокировки запроса.



```

# Файл для пользовательских правил WAF (Задание 2)
# Пример синтаксиса:
# SecRule REQUEST_HEADERS:User-Agent "@contains вредоносный_агент"
# "id:10000,phase:1,deny,status:403,msg:'Blocked Custom Scanner'"

SecRule REQUEST_HEADERS:User-Agent "@contains ZGrab/0.x"
"id:10007,phase:1,deny,status:403,log,msg:'Blocked scanner ZGrab/0.x'"

```

3. Скриншот вывода команды curl, демонстрирующий успешную блокировку запроса с подмененным User-Agent.

```

[anastasiacernoivan@MacBook-Pro-Anastasia lab2 % curl -i -A "ZGrab/0.x" http://127.0.0.1/
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Wed, 07 Oct 2026 06:51:04 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>

```

Задание 3 (Rate Limiting и L7 DDoS):

1. Заданные параметры rate и burst для целевого эндпоинта из варианта.

```
ab -n 500 -c 10 http://127.0.0.1/search
```

2. Фрагмент конфигурации из файла user_nginx.conf, содержащий добавленные директивы limit_req_zone и limit_req.

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=5r/s;
```

```
server {  
  
    listen 80;  
  
    server_name localhost;  
  
    location /search {  
  
        limit_req zone=mylimit burst=15 nodelay;  
  
        limit_req_status 503;  
  
  
        proxy_pass ${BACKEND};  
  
        proxy_set_header Host $host;  
  
        proxy_set_header X-Real-IP $remote_addr;  
  
        proxy_set_header X-Forwarded-For  
$proxy_add_x_forwarded_for;  
  
    }  
  
    location / {  
  
        proxy_pass ${BACKEND};  
  
        proxy_set_header Host $host;  
  
        proxy_set_header X-Real-IP $remote_addr;  
  
        proxy_set_header X-Forwarded-For  
$proxy_add_x_forwarded_for;  
  
    }  
}
```

```
}
```

3. Текстовый вывод результатов стресс-теста утилиты `ab` (или скриншот терминала), подтверждающий наличие ответов 503 Service Unavailable (показатель Failed requests или Non-2xx responses).

```
[anastasiacernoivan@MacBook-Pro-Anastasia lab2 % ab -n 100 -c 10 http://127.0.0.1/search > /dev/null; ]
curl -i http://127.0.0.1/search
HTTP/1.1 503 Service Temporarily Unavailable
Server: nginx/1.30.5
Date: Wed, 07 Oct 2026 07:10:29 GMT
Content-Type: text/html
Content-Length: 197
Connection: keep-alive

<html>
<head><title>503 Service Temporarily Unavailable</title></head>
<body>
<center><h1>503 Service Temporarily Unavailable</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
anastasiacernoivan@MacBook-Pro-Anastasia lab2 % █
```

Задание 4 (Проектирование профилей защиты):

1. Развернутый ответ на пункт 1: конфигурационные параметры Nginx для защиты от Slowloris (`client_body_timeout`, `client_header_timeout`, `keepalive_timeout`) с техническим обоснованием выбранных значений в секундах.

```
client_header_timeout 10s;

client_body_timeout    10s;

keepalive_timeout      15s;

send_timeout           10s;

reset_timedout_connection on;

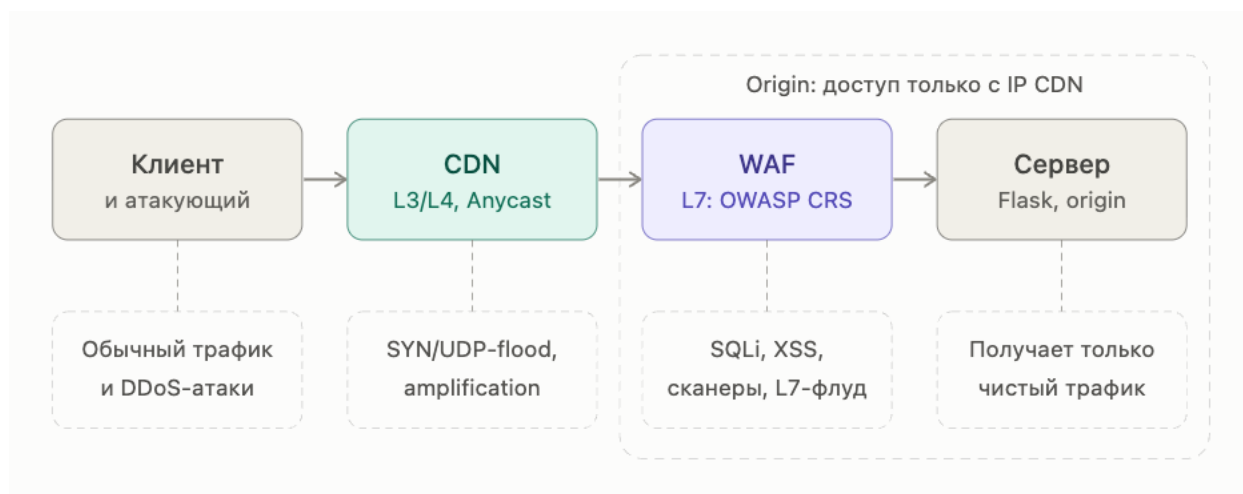

limit_conn_zone $binary_remote_addr zone=perip:10m;

limit_conn perip 20;
```

Обоснование:

- Атака держит соединения открытыми, медленно досылая заголовки или тело. Значение по умолчанию (60 с) слишком щедрое: 10 с достаточно для заголовков легитимного клиента даже при плохой связи, а медленные соединения быстро освобождают worker connections.
- `client_body_timeout 10s` задаёт паузу между чтениями тела, а не общее время загрузки, поэтому обычные загрузки не ломаются. Для эндпоинтов с большими загрузками значение можно поднять в отдельном location.
- `keepalive_timeout 15s` не даёт простаивающим соединениям занимать слоты.
- `limit_conn` ограничивает число одновременных соединений с одного IP, а `reset_timedout_connection on` сразу освобождает память сокета при таймауте.
- Слишком малые значения (1–2 с) приведут к ложным разрывам у пользователей с медленным каналом, поэтому выбран баланс 10–15 с.

2. Развернутый ответ на пункт 2: академическое описание и схема обработки трафика в эшелонированной защите Пользователь -> CDN (L3/L4 защита) -> WAF (L7 защита) -> Сервер приложения.



Принцип эшелонированной защиты:

- Объёмные L3/L4-атаки (десятки и сотни Гбит/с) гасит CDN: его канал и распределённая инфраструктура больше, чем у твоего сервера. Кэш статики дополнительно разгружает origin.
- WAF отсекает то, что CDN не видит по смыслу: SQLi, XSS, сканеры, перебор и L7-флуд.
- Origin должен быть скрыт: его IP не публикуется, фаервол пропускает на 80/443 только диапазоны CDN (при желании добавляется mTLS между CDN и WAF).
- Чтобы rate limiting на WAF работал по реальным клиентам, а не по IP CDN, настраивается `set_real_ip_from` с диапазонами CDN и `real_ip_header X-Forwarded-For`.

Вывод: в ходе работы был развёрнут стенд Nginx + ModSecurity с набором правил OWASP CRS: WAF заблокировал SQL-инъекцию, тогда как напрямую бэкенд принимал её, а кастомное правило `SecRule` остановило сканер `zGrab/0.x`. Rate limiting (5r/s, burst=15) на `/search` показал, что избыточные запросы L7-флуда отбрасываются с кодом 503 и бэкенд защищён от перегрузки, а проработанные профили против Slowloris и схема CDN + WAF закрывают медленные и объёмные атаки. Эшелонированная защита из нескольких механизмов оказалась эффективнее любого одиночного средства.